

Chapter 1

Study

1.1 Processor Design

The processors are all important components of computer architecture. Computer architecture is a specification that describes how hardware and software technologies are connected to create a computer platform. It refers to a system that processes any fetched instruction. There are many types of processor designs available and PPA tradeoffs are measured for each one of them and is then used in various applications. Instruction set architecture (ISA) defines program visible components and specifications it includes all the functions, capabilities, data formats, processor register etc. that are used by a computer system.

1.2 Instruction Set Architecture

ISA provides command to the processors, to tell what it needs to do. All processors can have same instruction set but still can have different internal design. In our project we will be using the RISC-V ISA. RISC-V instruction set architecture (ISA) is now set to become a standard free and open architecture for academic and industrial applications [?]. We have both 32-bit and 64-bit RISC-V designs, and in this project we will be going forward with 32-bit design.

The great interest on RISC-V comes from the fact that it is a free and open standard which allows anyone to easily develop their ideas. Furthermore RISC-V is modular, so it is based on a strong and frozen set, and all the extensions are optional without the need for continuous update as happens in incremental ISAs like the 80x86.

Comparison between various available ISA was made like MIPS, RISC, Intel, ARM [?]. RISC-V being an open source ISA, software development and ecosystem is available and the core can be used as an IP for various application development.

Despite the specific format used in the core we can identify four main subsets of instructions:

1. *Integer Computational Instructions:* They are encoded in R-type format or I-type format respectively, if we have a register-register operation or a register-immediate operation. We have addition (ADD, ADDI), subtraction (SUB), shifts (SLL, SLLI, SRL, SRLI, SRA, SRAI), bitwise logical operations (AND, ANDI, OR, ORI, XOR, XORI) and comparison operations (SLT, SLTI, SLTU, SLTIU).
2. *Control Transfer Instructions:* These instructions are responsible for jumps to other instructions in the range of ± 4 KiB that break the normal flow of the program. Jump And Link (JAL) and Jump And Link Integer (JALR) are dedicated to unconditional jumps and are encoded respectively with UJtype and I-type formats. Branch if Equal (BEQ), Branch if Lower Than (BLT), Branch if Lower Than Unsigned (BLTU), Branch if Not Equal (BNE), Branch if Greater or Equal (BGE), Branch if Greater or Equal Unsigned (BGEU). Note, BGT, BGTU, BLE, and BLEU can be synthesized by reversing the operands to BLT, BLTU, BGE, and BGEU, respectively.
3. *Load and Store Instructions:* These instructions are the only few simple instructions responsible for memory access. As RISC-V is a load-store architecture, the only memory related operations are loading a value from memory to a register and storing a value from a register to the memory. Register to Memory instructions are those of the STORE family (SW, SH, SB) that store respectively a Word (32b), Half a word (16b) and a Byte (8b). The instructions responsible for Memory to Register transfers are those of LOAD family (LW, LH, LHU, LB, LBU), where the U stands for Unsigned as the Half (16b) and Quarter (8b) words are zero-extended instead of sign-extended as in the non-U version.
4. *Control and Status Register Instructions:* These are all SYSTEM instructions, encoded using I-type format. Control and Status Register atomic Read/Write (CSRRW) is used to atomically swap values in the CSRs and integer registers; atomic Read and Set/Clear bits (CSRRS, CSRRC) are used to read the value of a CSR and then set or clear specific bits of the read word. The corresponding immediate versions of the three previous instructions, CSRRWI, CSRRCI, and CSRRSI, behave like their counterparts but they use an immediate instead of an integer register.

1.2.1 Standard extensions

In the previous paragraph we have briefly described the base integer ISA of RISC-V but as known the most important feature of RISC-V is the possibility to easily add other custom instructions to enlarge the ISA and adapt it to specific objectives. Despite these "non-standard" extensions, there are several other standard ISA extensions that have been developed during years and can be added very easily to our particular implementation making the final RISC-V a very powerful core, and they are:

- "M" Standard Extension for Integer Multiplication and Division;
- "A" Standard Extension for Atomic Instructions;

- "F" Standard Extension for Single-Precision Floating-Point;
- "D" Standard Extension for Double-Precision Floating-Point;
- "Q" Standard Extension for Quad-Precision Floating-Point;
- "L" Standard Extension for Decimal Floating-Point;
- "C" Standard Extension for Compressed Instructions;
- "B" Standard Extension for Bit Manipulation;
- "J" Standard Extension for Dynamically Translated Languages;
- "T" Standard Extension for Transactional Memory;
- "P" Standard Extension for Packed-SIMD Instructions;
- "V" Standard Extension for Vector Operations;
- "N" Standard Extension for User-Level Interrupts;

1.3 Microarchitecture

Majority of the processor designs have two components

1. Datapath
2. Control Path

Single cycle processor is a processor in which the instructions are fetched from the memory, and then executed, and the results are further stored in a single cycle, whereas in pipelined implementation, each step in execution takes one clock cycle. Since all instructions in a single cycle non pipelined structure do not necessarily take same amount of time rather the next instruction to be executed after an instruction can not start until the next clock cycle. Hence, the maximum clock frequency will be less and concurrent execution is not possible.

Pipelined processor saves time by overlapping instruction substages. Since, all RISC-V instructions are the same length, this makes it much easier to fetch instructions in the first pipeline stage and to decode them in the second stage. The more pipeline stages a processor has, the more instructions it can process at once and the less of a delay there is between completed instructions. The below table 1.1 shows the comparison between the two.

	Single Cycle	Pipelined Processor
Hardware Required	Less	More (Due to Registers)
CPI	1	≥ 1 (Due to Stalls and Hazards)
T_{clk}	Baseline	Less (Due to pipelining)
Throughput	1	> 1

Table 1.1: Comparison between Single cycle vs Pipelined

We will design a five stage pipelined architecture to achieve the benefit of maximum throughput and performance.

1. Instruction fetch
2. Instruction Decode
3. Instruction Execute
4. Memory
5. Write Back

1.4 Other available designs

Many processors based on RISC-V ISA has been developed and many are under progress. RISC-V is an open - sourced instruction set infrastructure that allows a new realm of processor creation by means of open collaborative domain, culminating in a pool of innovation with a 32- or 64-bit address space and a compressed ISA built for special application extensions [?].

Rocket [?] is a six staged open-source pipeline architecture with base as the RV64G ISA. The Rocket SOC generator is capable of generating multiple SOC design configuration prototypes from a single base instance. The Rocket scalar core, is a 64-bit 5-stage single-issue in-order pipeline that executes the RISC-V instruction set architecture (ISA).

Shakti-F series of processors are developed by a group of researchers from IIT Madras as an initiative to bring up computing units for space and nuclear research and development. Shakti-C, Shakti-T, Shakti-I, Shakti-E are variants of shakti computing core which are developed.

The RISC foundation serves as the foundation for ARM architecture, which similarly emphasizes simplicity and power economy. The load-store architecture, a combination of fixed-length 32-bit and variable-length Thumb instructions, and a significant amount of general-purpose registers are among ARM's core architectural features. These processors support cutting-edge functions including hardware virtualization, superscalar pipelines, and out-of-order execution [?].

	Rocket	Shakti	Current Design
Bits	32/64-bit	32/64-bit	32-bits
Stages	5	5	5
Extension	MAFD	IMAC	MAFD
Functional Units	4	3	5
Frequency	1.3Ghz(28nm,FD-SOI)	70Mhz (22nm,FinFet)	52Mhz(65nm)

Table 1.2: Application core comparative study

1.5 Previous RISC-V Architecture

Previous design of RISC-V Architecture is shown below in figure 1.1 with five stages of pipeline viz. Fetch, Decode, Execute, Memory and write back.

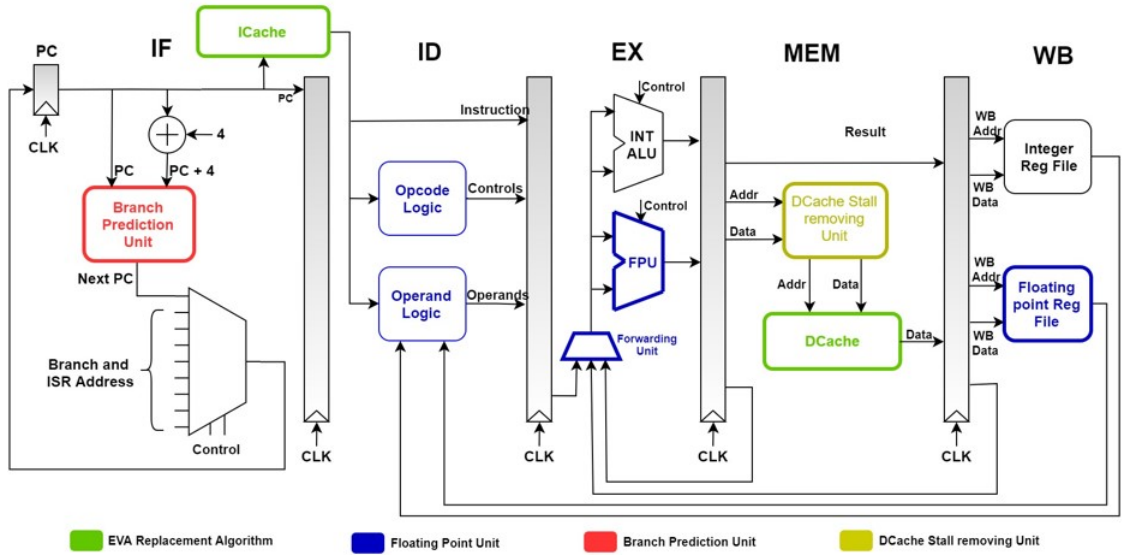


Figure 1.1: Previous Architecture

The baseline architecture, inherited from the previous batch, is a classic 5-stage pipelined RISC-V processor design consisting of the stages: *Instruction Fetch (IF)*, *Instruction Decode (ID)*, *Execute (EX)*, *Memory (MEM)*, and *Write Back (WB)*. Notably, the architecture integrates several performance-enhancing features. In the IF stage, it incorporates a *Branch Prediction Unit* to reduce branch misprediction penalties and a dedicated *Instruction Cache (ICache)* to accelerate instruction fetch latency. During decoding, control signals are generated by Opcode and Operand Logic, which interface with the Forwarding Unit in the EX stage to handle data hazards. The Execute stage features both an *Integer ALU* and a *Floating Point Unit (FPU)* for supporting mixed-type operations, reflecting floating-point support across the pipeline. The MEM stage includes a *Data Cache (DCache)* to reduce data memory access times, and a DCache Stall Removing Unit, which aims to improve memory pipeline throughput by mitigating stalling due to cache misses. Finally, the WB stage supports both integer and floating-point register

files, highlighting the design's ability to execute and store results from a wide range of computational instructions. This architecture served as a strong foundation for further optimizations and extensions in the current work.

1. ***Instruction fetch:*** At the start of each cycle, the address is used to fetch instructions from instruction memory. Following that, a calculation is run to compute the next PC in the cycle. It is decided either to use that as its next value or, instead, the outcome of the control transfer instruction based on branch prediction logic.
2. ***Instruction Decode:*** The 32-bit instruction is parsed into its fields, and two registers pertaining to the operands are fetched from the register file. Instruction mux, Instruction decoder, Immediate generator, Register file, and the write enable generator are the several blocks that make up the instruction decode stage.
3. ***Instruction Execute:*** In the Execute step, the real computing takes place. An ALU and a bit shifter are frequently included in this phase. The bit shifter is in charge of shifts and rotation logic to be performed on the operands.
4. ***Memory Access:*** At this point, the load/store instructions read and write memory operands from and to the memory.
5. ***Write-Back:*** At this step, the instructions store their results to the respective register file. There will be an onset of data hazard since one of the input registers read in decode could be the same as the destination register written in writeback. The data forwarding technique is used to avoid this.

1.5.1 Hazard Detection and Correction block

Due to data and control dependencies, in pipeline designs we have 2 major hazards:

1. *Data Hazard*
2. *Control Hazard*

When an instruction in the pipeline is dependent on the outcomes of a preceding instruction that has not yet entered the write back stage, it leads to data dependence. Control dependency occurs when control instructions are executed. The choice to branch or not branch for these instructions is made during the execution stage, after two consecutive instructions have entered the pipeline's fetch and decode stages. If no branch is chosen, the execution continues uninterrupted. However, if the branch is taken, the prior two instructions must be drained out before the current branch instruction can be executed, resulting in a control hazard.

1. *Data Forwarding to eliminate data hazard:* Ahead of the source instruction reaching write back stage, data is made available to the ALU for dependent instructions, which is known as forwarding. This data forwarding technique identifies the dependant instruction in pipeline and forwards the data to the dependant instruction thus

eliminating data hazards in the pipeline. *We'll be using both IE-IE and Mem-IE data forwarding.*

2. *Adaptive dynamic Branch prediction to eliminate control hazard:* It enhances the accuracy of prediction by taking into account the recent behaviour of previous branches as well. If the instruction is a branch instruction it uses Branch Target Address, which is fetched during the Instruction fetch stage. It also makes use of a Local Prediction Table, which contains two bits that decide whether or not a branch is taken. In comparison to a one bit branch predictor, the branch predictor state changes only after two consecutive mispredictions, resulting in enhanced accuracy.

Figure 1.2 shows the Fully bypassed datapath, which is used to bypass the result either from the IE/MEM stage or from MEM/WB stage based on the next instruction after decode stage.

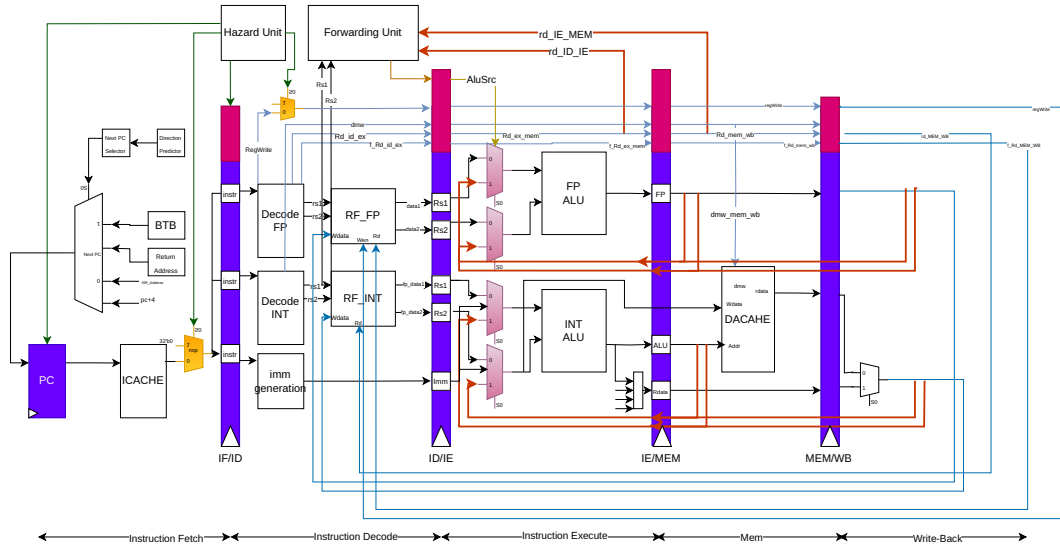


Figure 1.2: Bypass Datapath

If the current instruction is a ALU and ALUi type then we only need bypass and not stall. But if we have LD, JAL, JALR as our current instruction, then we need to stall the next instruction for 2 clock cycles. After 2 clock cycle, these instructions will be in the WB stage and hence we can then use the result of current instruction as an operand for the next waiting instruction. For the 2 cycle the stall signal will be asserted which will hold the instructions.

1.5.2 Branch Prediction Unit

When a program runs certain flow of instruction is fetched and executed in pipeline. However, when a branch instruction is fetched, it may change the flow of the instructions. The execution result of branch instructions determines whether to change the flow in the

execution stage, which is in the middle of the pipeline stage. If it changes its flow, the next instructions that are already being executed by other hardware resources prior to the execution stage become useless, which leads to a pipeline flush and degrades the processing speed. Since the branch instructions take more than 20 % of all instructions when running a program, this pipeline flush becomes a huge performance degradation factor in processors.

Branch prediction unit contains three different blocks: **branch direction predictor**, **branch target buffer (BTB)** and **Return Address Stack**. When instruction is fetched, branch prediction unit will identify whether it is branch instruction. If the instruction is branch, the direction predictor will give the prediction result (Taken / Not Taken). If the branch is taken then the PC is updated by the target address obtained from BTB. The branch prediction unit is usually placed in instruction fetch stage. So, when prediction is correct the next instruction fetched after the branch instruction is from the correct instruction flow and pipeline can execute these instruction without any pipeline flush. But when prediction is wrong, the instruction executed after the branch needs to be flushed.

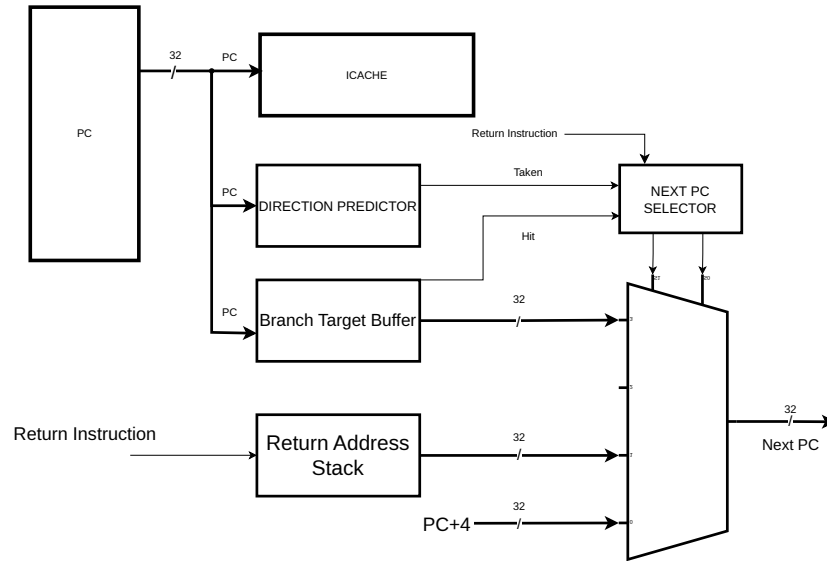


Figure 1.3: Branch Prediction Unit

The Branch Prediction Unit (BPU) in this architecture includes a Return Address Stack (RAS), which is specifically used to predict the target of function return instructions (ret). The RAS stores the return addresses pushed during function calls (jal or jalr to ra) and pops them during returns, thereby improving prediction accuracy for nested and recursive function calls. This mechanism helps in maintaining pipeline flow and reducing control hazards caused by subroutine returns, especially in deep call hierarchies.

1.5.3 ICACHE

The instruction cache (ICache) implemented in this architecture is a 2-way set-associative cache, designed to enhance instruction fetch performance by reducing cache misses compared to a direct-mapped design. This configuration divides the cache into multiple sets, with each set containing two lines (or "ways"), allowing the processor to store multiple blocks that map to the same index, thereby minimizing conflicts. The use of a 2-way associative structure strikes a balance between hardware complexity and hit rate efficiency, ensuring faster access to instructions while maintaining manageable control logic. This setup is particularly beneficial in pipelined processors where continuous and rapid instruction supply is critical to maintain throughput and minimize stalls.

1.5.4 DCACHE

The data cache (DCache) used in this design is an 8KB, 2-way set-associative, write-back cache, optimized for efficient memory access and reduced latency in load/store operations. The 2-way set-associative architecture divides the cache into sets with two lines per set, offering a good compromise between simplicity and performance by reducing conflict misses when multiple memory blocks map to the same index. The write-back policy employed ensures that write operations are first made to the cache, and only written back to main memory when a cache line is replaced. This reduces memory traffic and improves overall performance, especially in programs with frequent data modifications. The DCache is tightly integrated into the pipeline to support fast and efficient data access, contributing significantly to the overall execution speed of the processor.

1.6 Proposed architecture

The earlier RISC-V processor architecture was designed with a fundamental execution pipeline and support for the base IMA extensions—Integer, Multiplication/Division, and Atomic operations. While this core provided a functional and efficient execution platform, it lacked several critical features necessary for full compliance with the RISC-V privileged architecture specifications. Notably, the previous design did not support CSR (Control and Status Registers), exception and interrupt handling, debug infrastructure, memory protection mechanisms (PMP), and address translation for virtual memory. These missing components limit its practical deployment, especially in real-world operating systems and secure embedded systems.

To address these limitations, our proposed architecture introduces a fully compliant RISC-V core with support for all major privileged functionalities. The processor will be enhanced with modules for **CSR** management, exception and **interrupt handling**, **debugging support**, TLB for virtual memory translation, and cache control features such as flushing and invalidation. We also aim to integrate Physical Memory Attributes (PMA) and **Physical Memory Protection (PMP)** units for enforcing access control and memory policies. The development of this architecture will **culminate in a cus-**

tom ASIC implementation, transitioning the design from FPGA prototyping to a silicon-based deployment for performance and power optimization.

1.6.1 Control and Status Registers (CSR)

CSR is an integral part of the RISC-V architecture that provides control over various processor functionalities such as performance counters, trap handling, privilege levels, and configuration registers. In the new architecture, a dedicated CSR block is implemented to handle both read and write operations in a single pipeline cycle. The CSR module decodes instructions like `csrrw`, `csrrs`, and `csrrc` to interact with specific registers. It interfaces closely with the exception/interrupt controller and privilege management, enabling dynamic control of processor behavior across different modes (User, Supervisor, Machine).

1.6.2 Exception and Interrupt Handling

Robust exception and interrupt handling is critical for reliable system behavior and compliance with operating system requirements. The new design includes a centralized trap controller that supports synchronous exceptions (e.g., illegal instruction, load/store faults) and asynchronous interrupts (e.g., timer, external). It manages control flow redirection using the `mepc`, `mcause`, and `mtval` CSRs and distinguishes between privilege levels for servicing each event. The design ensures deterministic behavior through prioritization and vectored interrupt support, enabling the processor to handle real-time tasks and secure environments.

1.6.3 Debug Infrastructure

The debug module introduces capabilities for runtime inspection, single-step execution, breakpoints, and system halting. This infrastructure is essential for embedded system developers and operating system debugging. Our design will conform to the RISC-V Debug Specification, integrating a Debug Module Interface (DMI) and halt/exception debug control. Features include program buffer execution, abstract register access, and interaction with external debug tools through JTAG or other interfaces. The debug system operates independently of normal execution flow, allowing inspection without altering program behavior.

1.6.4 Translation Lookaside Buffer (TLB) with Two-Level Page Table

To support virtual memory and operating system-level address translation, a Translation Lookaside Buffer (TLB) is introduced. The TLB stores recent translations from virtual to physical addresses, significantly accelerating memory access. Our TLB design uses a

two-level page table model compliant with the Sv32 and Sv39 schemes defined in RISC-V. This allows efficient address translation for 32-bit and 64-bit systems, with hardware page walkers to fetch entries from page tables on TLB misses. The TLB is tightly integrated with memory management and privilege checking to enforce access rights and improve performance.

1.6.5 Cache Flushing and Invalidation

To maintain coherence and correctness in multi-threaded or I/O-intensive applications, the architecture includes support for cache flushing and invalidation. These mechanisms are implemented through specialized CSR instructions (like fence.i, custom flush, or invalidate commands) that target instruction and data caches. Cache control is particularly important during context switches, self-modifying code, or DMA transfers. Our design ensures that cache operations are synchronized with pipeline execution to avoid hazards and support safe memory sharing.

1.6.6 Physical Memory Attributes (PMA) and Physical Memory Protection (PMP)

Security and memory access control are fundamental for modern embedded systems. To this end, the architecture integrates PMA and PMP blocks. PMP enables software to define access permissions for memory regions, specifying whether a range is readable, writable, executable, or inaccessible at a given privilege level. This forms the foundation for isolating processes and protecting critical system resources. PMA, on the other hand, defines hardware attributes of memory regions (e.g., cacheable, bufferable), allowing the processor to handle various types of devices and memory-mapped I/O safely. These mechanisms are configured via CSRs and tightly coupled with the MMU and TLB.

1.7 ASIC Flow

ASIC (Application Specific Integrated Circuit) is a specialized type of integrated circuit meticulously designed to perform a specific function or set of functions within an electronic system. ASICs are tailor-made for a particular application, offering unparalleled efficiency and performance. There are two primary methods of ASIC design:

1. *Gate Array (Semi Custom) design* : This type of ASIC uses predesigned logic cells called standard cells, such as gates, multiplexers, and flip-flops. Standard cells are made using full-custom design methodology and serve as basic building blocks for ASIC design, ensuring the same performance and flexibility but reducing time and risk.
2. *Full Custom design*: The full-custom method is more complex and costly, but it can do much more than the gate array method. The size of the ASIC decreases

significantly as the design incorporates only the necessary gates and electronics, and unused gates are deleted. These ASICs are designed for a specific purpose and support a particular function in the end product.

1.7.1 Chip Design flow

A typical design flow follows a structure shown below 1.4. Some of these phases happens parallelly and some sequentially.

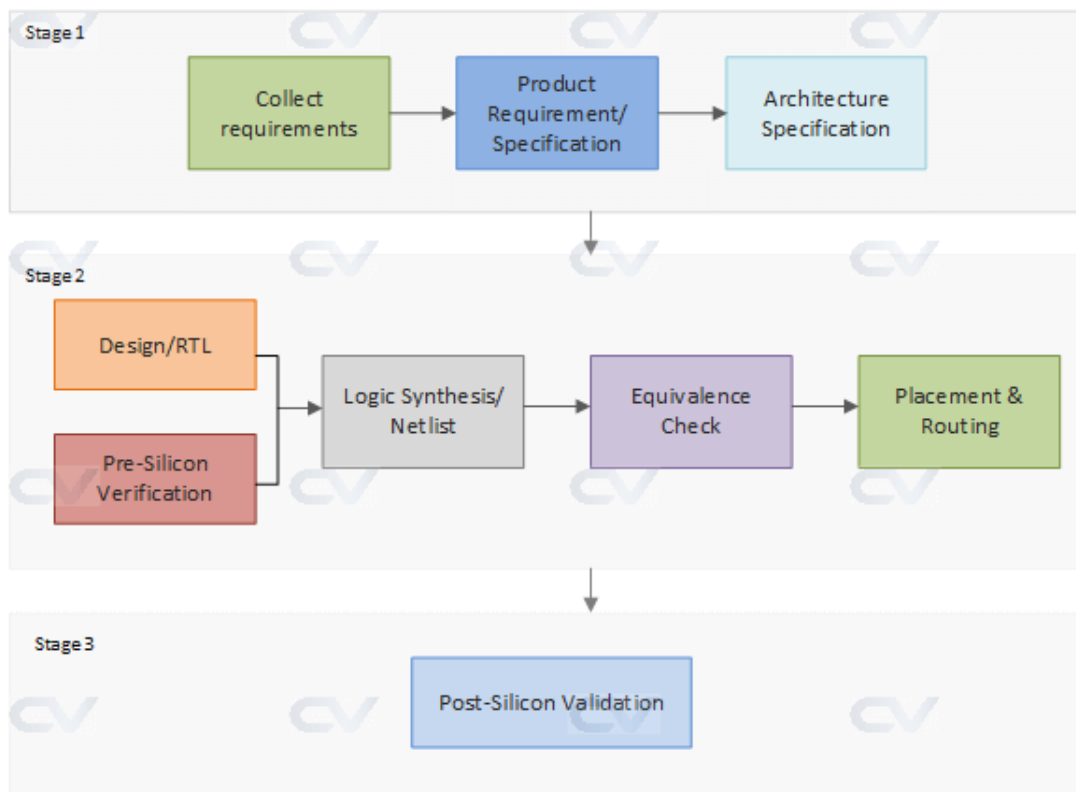


Figure 1.4: Chip Design Flow

Architecture: The Architects based on the requirement specifications and market need, comes up with a system level view of how the chip should operate. They will decide what components to use, on which frequency should the chip run, what should be the footprint, operating voltages and estimate the power numbers. They also decide how will be the dataflow inside the chip

Digital Design: Since the designs are complex nowadays, we cannot write RTL from scratch and hence various IP's are taken and integrated into the design. A behavioral description of the blocks are written in HDL and the design is analyzed in terms of functionality, performance and other high level issues.

Verification: Once the RTL is ready, it needs to be verified for functional correctness. Various EDA tools like simvision and xcellium (from Cadence), Verdi (from Synopsys) are

used for verifying the RTL. Currently, Verification has become an important part of chip designing and is the most time consuming process in the lifecycle. Various methodologies like UVM, OVM have been adopted for better verification of the design and to reduce time to market

To save time and reach functional closure, both the design and verification teams operate in parallel where the designers “release” the RTL version, and the verification team develops the testbench environment and test cases to test the functionality of the RTL Version. A UVM scoreboard is shown in figure 1.5 below.

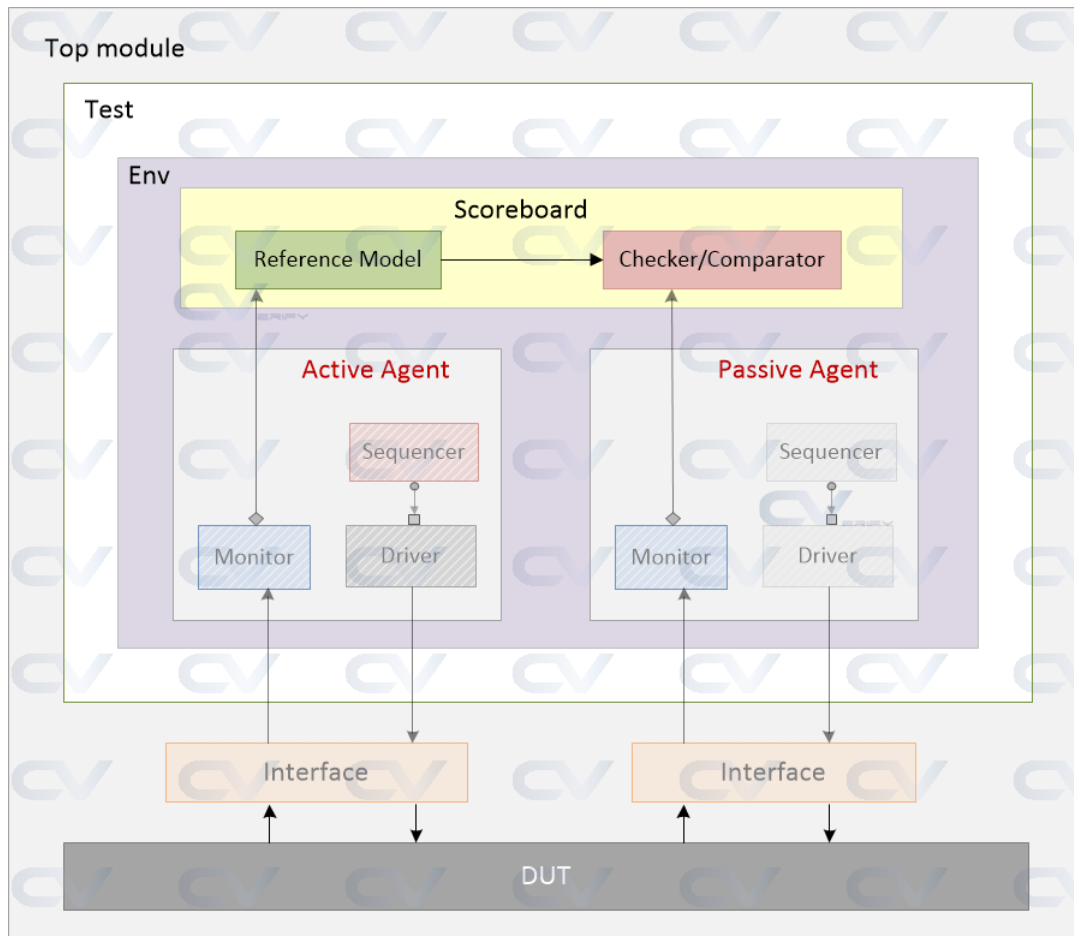


Figure 1.5: UVM Scoreboard

Logic synthesis and Equivalence check: Once the design is ready and verified, we convert it into hardware schematic with real elements like combinational gates, muxed, FlipFlops. Logic synthesis tool like Genus (from Cadence), enable us to convert RTL description into a gate level netlist. Tool ensures the netlist meets the timing, area and performance. We have the library files for each element in the netlist which we get from foundaries as PDKs.

The gate level netlist is checked for logical equivalence with the RTL and GLS (Gate level Simulations) are required to ensure the synthesized design still holds the same functionality. Genus GUI is shown in figure 1.6 below.

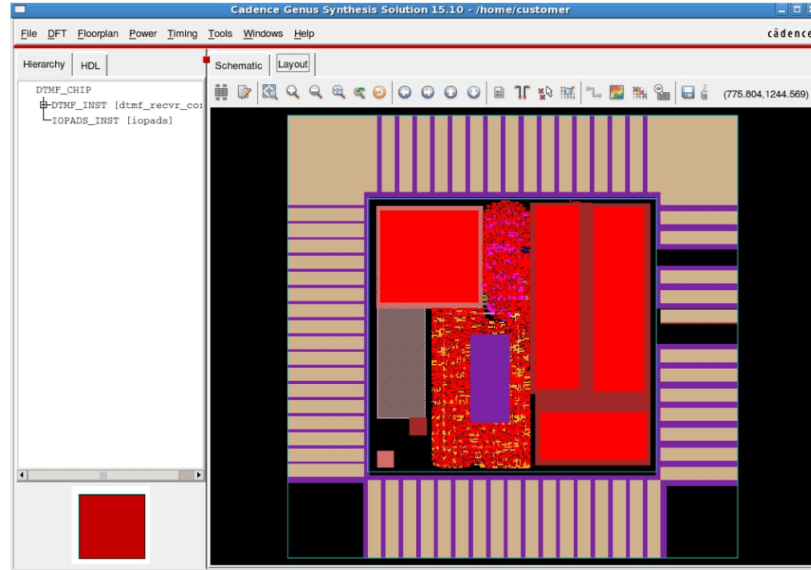


Figure 1.6: Genus Tool for Synthesis

Placement and Routing: The netlist is sourced into the physical design flow, where automatic place and route (APR or PnR) is done with the help of EDA tools, Eg. Cadence Innovus, Encounter and Synopsys IC compiler. This will select and place standard cells into rows, define ball maps for input-output, create different metal layers, and buffers to meet timing. Once this process is done, the layout is send for fabrication. Cadence Encounter GUI is shown in below figure 1.7.

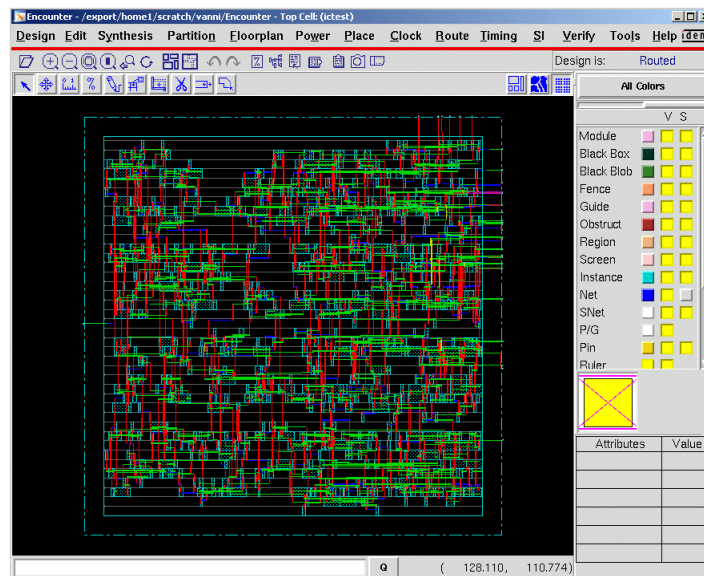


Figure 1.7: Cadence Encounter

1.8 JTAG (IEEE 1149.1)

The Joint Test Action Group (JTAG), formalized as the IEEE 1149.1 standard, provides a standardized serial boundary-scan architecture. It is primarily intended for structural testing, in-system programming, and embedded debugging of digital integrated circuits (ICs). JTAG has evolved into an indispensable interface, especially within modern System-on-Chip (SoC) and FPGA environments, enabling non-intrusive access to internal logic and registers. This capability facilitates design validation, post-silicon verification, board bring-up, and runtime diagnostics, making JTAG a cornerstone in contemporary digital design workflows.

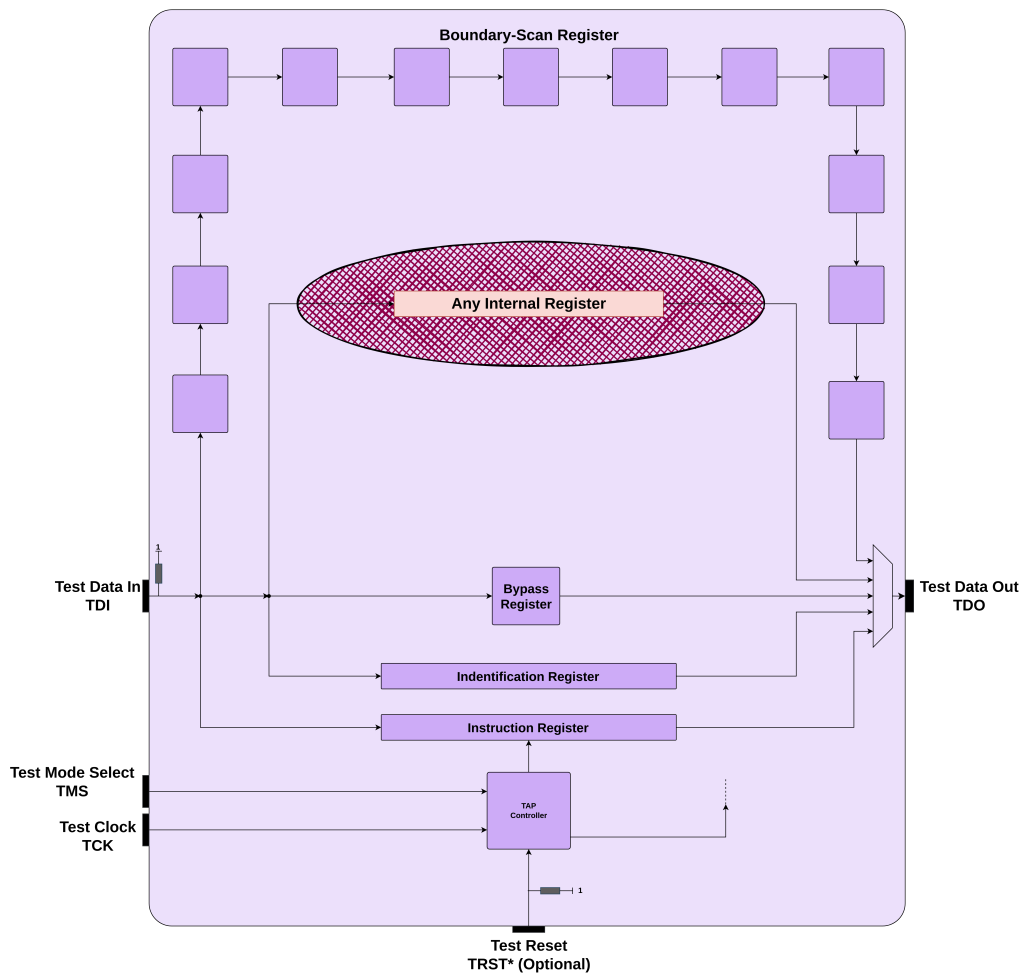


Figure 1.8: IEEE 1149.1 Chip Architecture

1.8.1 JTAG Signal Description

- **TDI (Test Data In):** A unidirectional serial input line through which instruction or test data is injected into the target device. In multi-device configurations, it typically connects to the TDO of the preceding device.

- **TDO (Test Data Out):** The serial output line that transmits the response data from the device. It is chained to the TDI of the subsequent device.
- **TCK (Test Clock):** A dedicated clock signal for synchronizing all JTAG operations. It is independent of the system clock, enabling asynchronous test access.
- **TMS (Test Mode Select):** Governs transitions within the TAP controller's state machine. The state of TMS is sampled on the rising edge of TCK.
- **TRST (Test Reset – optional):** An asynchronous reset signal that forces the TAP controller into the Test-Logic-Reset state without requiring TCK activity.

1.8.2 JTAG Timing Diagram

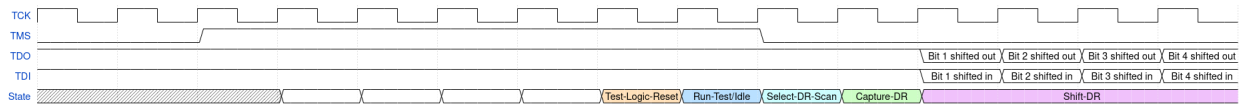


Figure 1.9: JTAG Timing Diagram

JTAG timing diagrams describe the temporal coordination between TDI, TDO, TMS, and TCK during scan operations. Key aspects include:

1. **Setup and Hold Requirements:** Data inputs TDI and TMS must remain stable relative to the rising edge of TCK to ensure deterministic sampling.
2. **TDO Validity:** Output data becomes valid on the falling edge of TCK and must be captured externally before the next transition.
3. **Data and Instruction Shifting:** Precise clocking ensures successful bit-wise propagation through instruction and data registers.

Robust timing analysis becomes increasingly critical as scan chains grow in length or operate at higher frequencies.

1.8.3 Test Access Port (TAP) Overview

The Test Access Port (TAP) acts as the primary interface between external JTAG tools and the internal infrastructure of the target integrated circuit. It establishes a serial communication path that enables access to the Instruction Register (IR), various Data Registers (DRs), and the TAP Controller's state machine. Through this interface, test patterns can be delivered, internal data can be acquired, and control signals can be propagated—all without the need for additional I/O resources. Owing to its minimal pin requirements and standardized operation, the TAP plays a critical role in facilitating embedded debugging, hardware validation, and boundary scan operations within modern digital systems.

1.8.4 TAP Controller State Machine

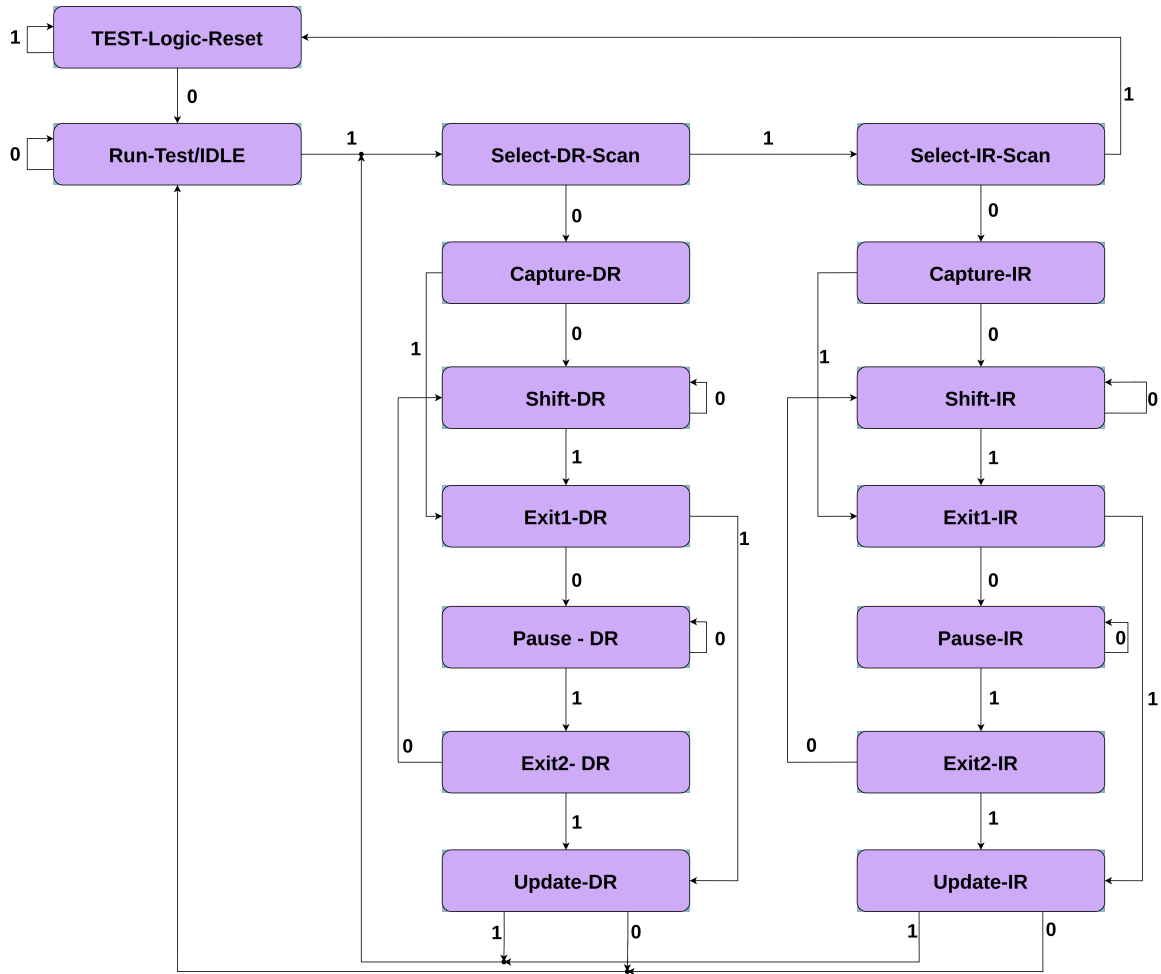


Figure 1.10: TAP Controller State Machine

At the heart of the JTAG interface lies a deterministic 16-state finite state machine known as the Test Access Port (TAP) controller. This controller orchestrates the sequencing and execution of all JTAG operations by controlling access to the internal instruction and data registers of the target device. The TAP controller is central to enabling flexible test and debug functionality over a minimal pin interface. Its primary states are outlined below:

- **Test-Logic-Reset:** This state asynchronously initializes all JTAG logic within the device, ensuring that the system enters a well-defined, inactive configuration. It is commonly used to recover from an unknown or unstable state before initiating a test or debug session.
- **Run-Test/Idle:** After reset, the TAP enters this state, which either maintains the system in a passive idle condition or triggers the execution of specific test operations, such as Built-In Self-Test (BIST) or continuous scan testing, depending on the configuration of the active instruction.

- **Select-DR-Scan / Select-IR-Scan:** These intermediate states determine the type of register path—Data Register (DR) or Instruction Register (IR)—that will be accessed next. They serve as branch points for directing the TAP controller’s flow toward the desired scan operation.
- **Capture, Shift, Pause, Update States:** These sets of states exist for both DR and IR scan paths and are responsible for the detailed handling of serial data. The Capture state samples current values into the respective shift register; Shift allows for serial movement of bits through the register via TDI and TDO; Pause temporarily halts shifting to allow external processes or decision-making; and Update writes the shifted data into the active register, committing the operation.

State transitions within the TAP controller are governed by the Test Mode Select (TMS) signal, which is evaluated on the rising edge of the Test Clock (TCK). This serialized control scheme enables deterministic navigation through the TAP state machine using a single control line, eliminating the need for parallel signaling or complex external logic. Such a mechanism ensures precise coordination of test operations and register access.

1.8.5 Instruction Register

The Instruction Register (IR) defines the operational context of the JTAG interface by determining which data register is accessible during a given operation. Typically 5 bits wide, the IR is loaded serially via the Shift-IR and Update-IR states within the TAP controller. Standardized instructions include:

- **EXTEST:** Enables external pin-level testing through the boundary scan register.
- **SAMPLE/PRELOAD:** Captures the real-time state of selected signals or preloads values into scan chains for subsequent operations.
- **IDCODE (optional):** Provides a static device identifier to facilitate device recognition and configuration. If the identification register is not implemented in hardware, the IDCODE instruction must be interpreted as a BYPASS instruction.
- **BYPASS:** Selects a minimal 1-bit data register, allowing the device to be bypassed in a scan chain and enabling faster access to downstream devices.
- **USER Instructions:** Reserved opcodes that support vendor-defined or application-specific functionality, enhancing the flexibility of the JTAG interface.

The support for USER instructions significantly extends the utility of the IR by enabling custom debug features and facilitating proprietary extensions within standardized test workflows.

1.8.6 Data Registers Overview

Data Registers (DRs) in the JTAG interface are used for data exchange between the external debugger and the target device. The active DR is determined by the current instruction loaded into the Instruction Register (IR), with different instructions providing access to various DRs. These registers allow operations such as reading and writing to internal registers, configuring device state, and performing other test and debug operations.

Data is shifted serially through the DRs during scan operations, enabling external debuggers to interact with the processor's internal state. This mechanism supports a range of tasks, including control, inspection, and configuration, all facilitated by the JTAG scan protocol. The ability to select different DRs based on the loaded instruction adds flexibility, enabling tailored debug and testing functionality specific to the target device's needs.

1.8.6.1 IDCODE Register

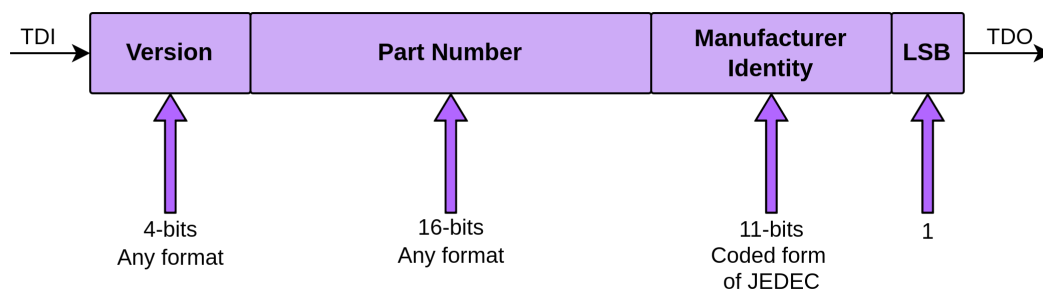


Figure 1.11: Device Identification Code Structure

The IDCODE register is a 32-bit read-only register that provides a standardized identifier for the target device. Its structure is as follows:

- 4 bits for Version Number
- 16 bits for Part Number
- 11 bits for Manufacturer ID
- 1-bit Stop Bit (logic high)

The IDCODE register plays a critical role in automated toolchains by enabling target detection, configuration validation, and the provisioning of firmware. It facilitates device identification and ensures compatibility between the debugging tools and the target hardware, making it an essential component for efficient debugging and system management.

1.8.6.2 BYPASS Register

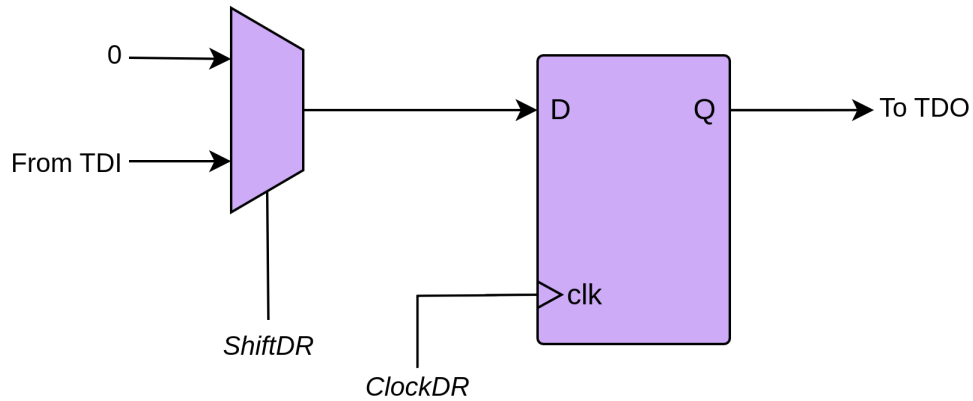


Figure 1.12: Bypass Register Structure

The BYPASS register is a one-bit shift register that serves to exclude a device from the scan chain when it is not the target of a JTAG operation. This functionality optimizes the scan chain by reducing latency and simplifying data transfers in multi-device configurations. When the device is in the bypass state, JTAG signals pass through it without interacting with its internal logic, enhancing the efficiency of the test and debug processes.

1.8.6.3 Boundary Scan Register (optional)

The Boundary Scan Register (BSR) is an optional but highly valuable feature that enables access to the device's digital I/O pins through a shift register architecture. It facilitates a range of critical functions:

- **Non-intrusive interconnect testing:** The EXTEST instruction utilizes the BSR to test the device's interconnects without the need for physical probing, ensuring integrity without disrupting the system.
- **Real-time pin state capture:** Through the SAMPLE instruction, the BSR allows for the real-time capture of I/O pin states, providing a reliable method for observing device behavior during testing.
- **Controlled input driving:** The PRELOAD instruction enables the controlled driving of input signals, which is essential for performing diagnostics at the board level.

Its inclusion significantly enhances the observability and controllability of physical signal paths without requiring intrusive probing.

1.8.6.4 Implementation-Specific Data Registers

In advanced system designs, custom data registers are often implemented to support specialized functions tailored to the specific needs of the device. These registers may include:

- **Debug module communication:** Registers designed to facilitate efficient communication with on-chip debug modules, enabling detailed control and monitoring during the debugging process.
- **Memory or CSR (Control and Status Register) access:** Registers that provide direct access to memory regions or critical system control registers, enhancing low-level interaction with the system's internal state.
- **Performance monitoring:** Custom registers dedicated to tracking system performance metrics, providing insights into operational efficiency, resource utilization, or bottlenecks.

These user-defined data registers significantly enhance the flexibility and functionality of the JTAG interface, particularly in FPGA or processor-centric systems, where seamless integration with on-chip resources is essential for effective operation and debugging.

1.8.7 Supported JTAG Instructions

JTAG-compliant devices implement a core subset of instructions as defined by the IEEE 1149.1 specification. These instructions govern the configuration of the scan path and determine which data register is active during JTAG operations. Some of the most common instructions include:

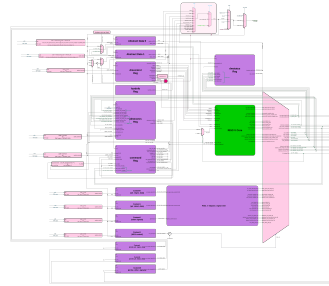
- **BYPASS:** Activates the 1-bit bypass register.
- **IDCODE:** Enables the ID register for device identification.
- **EXTEST:** Facilitates external connectivity testing.
- **SAMPLE/PRELOAD:** Provides mechanisms for input capture or pattern setup.
- **USER Instructions:** Enable access to implementation-specific logic.

Instruction decoding and register mapping form the foundation of low-level test and debug access in JTAG-based systems. The flexibility to define and integrate custom instructions is a key feature of the JTAG architecture, ensuring that it can be adapted to meet the requirements of a wide range of applications, from basic device testing to complex, domain-specific debug scenarios.

In summary, the JTAG interface is an essential tool for test, debug, and in-system configuration in complex digital systems. Its well-structured state machine, standardized signaling protocol, and extensible register architecture make it a powerful mechanism for verification, post-silicon debugging, production testing, and ongoing system maintenance.

1.9 RISC-V Compliant Debugger

1.9.1 Overview of RISC-V Debug Specification



RISCV Debug System Overview

The RISC-V Debug Specification defines a modular and transport-agnostic framework for external debuggers to interact with RISC-V processors. Its architecture separates transport, control, and execution layers to provide scalable and flexible debug support. The primary components include the Debug Transport Module (DTM), the Debug Module Interface (DMI), and the Debug Module (DM), each fulfilling distinct roles in the debug pathway.

Debug communication typically begins with a debugger host (e.g., GDB) interfacing with a hardware driver (e.g., OpenOCD), which interacts with the Debug Transport Module (e.g., JTAG or UART). The DTM then relays these instructions to the Debug Module via the DMI, which exposes control/status registers and enables register-level operations on the processor core (hart). This architecture supports both single and multi-hart environments.

1.9.2 Debug System Components

The RISC-V debug architecture is modular, consisting of three primary components: the Debug Transport Module (DTM), the Debug Module Interface (DMI), and the Debug Module (DM). These components work in conjunction to facilitate interaction between an external debugger and the internal state of one or more RISC-V harts.

- ***Debug Transport Module (DTM)***: Acts as the physical access point for an external debugger. It implements a protocol-specific interface—typically JTAG or Serial Wire Debug (SWD)—and forwards incoming debug transactions to the Debug Module through the DMI.
- ***Debug Module Interface (DMI)***: Serves as a memory-mapped interface between the DTM and the Debug Module. It supports 32-bit read and write transactions to a defined address space, enabling command and status exchange. The interface is intentionally minimalistic to ensure lightweight integration with transport mechanisms.

- **Debug Module (DM):** Implements the primary debugging functionality. It controls hart execution (halt, resume), provides abstract access to memory and registers, and supports programmable instruction execution using a program buffer. A single DM typically serves all harts in a system, allowing fine-grained control via internal hart selection mechanisms.

Each hart (hardware thread) is associated with one DM. In multi-hart systems, a centralized DM facilitates debugging across all harts, simplifying design and reducing overhead.

1.9.3 Debug Module Interface (DMI)

The DMI provides the communication pathway between the JTAG interface (via the DTM) and the internal Debug Module. It exposes a 32-bit addressable register space where debug operations can be initiated or queried. The interface is streamlined, requiring only address and data lines, enabling efficient read/write access without additional control handshaking. This simplicity supports both hardware and software implementations with minimal complexity.

1.9.4 Debug Module Registers

The Debug Module exposes several memory-mapped registers via DMI to configure, control, and monitor debugging operations. Key registers include:

- ***dmcontrol* (0x10):** Enables the debug module, selects the target hart, and issues halt/resume requests.
- ***dmstatus* (0x11):** Reflects the current status of all managed harts (e.g., halted, running, unavailable).
- ***abstractcs* (0x16):** Indicates the availability and configuration of abstract command mechanisms, such as the program buffer.
- ***command* (0x17):** Encodes abstract commands to be executed by the DM (e.g., register access).
- ***abstractauto* (0x18):** Automates execution of abstract commands and data access patterns.
- ***data0–dataN* (0x04–0x0f):** Temporary storage registers for abstract command operands or results.
- ***progbuf0–N* (0x20–0x2X):** Holds RISC-V instructions to be executed during debug mode using the program buffer.

Additional registers such as *hartinfo*, *nextdm*, *dmcs2*, and implementation-specific extensions can be included to support more advanced functionality or multicore management.

1.9.5 Abstract Commands and Program Buffer

Abstract commands provide a flexible, implementation-independent method for executing fundamental debug tasks. The DM supports the following command types:

- ***Access Register:*** Read or write general-purpose registers (GPRs) or control/status registers (CSRs).
- ***Quick Access:*** Perform a rapid sequence of halt, instruction execution, and resume for common diagnostics.
- ***Access Memory:*** Read or write system memory from the perspective of the target hart.

For more complex operations, a Program Buffer allows execution of short RISC-V instruction sequences. The buffer typically supports at least two instruction slots and must terminate with an ebreak (or c.ebreak) instruction. This mechanism allows custom register transfers, CSR manipulation, and even limited branching within debug mode.

1.9.6 Hart Debug Control & CSRs

Each hart features dedicated control and status registers for managing debug state:

- ***dcsr (0x7b0):*** The Debug Control and Status Register includes fields indicating the cause of entry into debug mode, the current privilege level, single-step enable, and other configuration bits.
- ***dpc (0x7b1):*** Debug Program Counter stores the address at which the hart was halted.
- ***dscratch0, dscratch1 (0x7b2, 0x7b3):*** General-purpose scratch registers available for temporary storage by the debugger during execution of abstract commands or program buffer instructions.

These CSRs ensure safe and controlled operation of the debugger and allow seamless entry and exit from debug mode.

1.9.7 Debug Mode Operation

A RISC-V hart enters Debug Mode when specific events or conditions occur that indicate external or internal debug intervention. These conditions include:

- ***External Halt Request:*** The debugger sets the haltreq bit in the dmcontrol register via the Debug Module Interface (DMI). This signal requests that the hart halt at the next instruction boundary.

- **Trigger Events:** The hart is configured with hardware breakpoints or watchpoints (via trigger modules). Upon matching a trigger condition—such as a specific instruction fetch, memory access, or address match—the hart automatically transitions to Debug Mode.
- **Single-Step Execution:** If the step bit is set in the dcsr (Debug Control and Status Register), the hart executes one instruction and then automatically halts and enters Debug Mode. This is useful for instruction-level stepping during software debugging.

Once a hart enters Debug Mode, the following changes occur in its execution environment:

- **Instruction Fetch Is Disabled:** The hart’s normal instruction execution pipeline is stalled. It does not fetch or retire instructions from the normal program memory, effectively freezing the program state.
- **Access to Debug Resources:** While halted, the Debug Module gains exclusive access to the hart’s internal state. Abstract commands issued by writing to the command register allow the debugger to read or write general-purpose registers (GPRs), control and status registers (CSRs), or memory.
- **Execution via Program Buffer:** A small, fixed-size instruction buffer (progbuf) inside the Debug Module allows the debugger to inject and execute RISC-V instructions while the hart is halted. The program buffer enables flexible, direct manipulation of the hart state (e.g., loading registers from memory, executing computation, etc.). For correct behavior, program buffer execution must end with an ebreak or c.ebreak instruction, which causes control to return to the Debug Module.
- **Preservation of Program State:** Upon entering Debug Mode, the current value of the Program Counter (PC) is saved in the dpc register. The cause of entry is recorded in dcsr.cause, which can indicate external request, trigger match, single-step, or exception during debug execution. The dcsr also reflects the previous privilege level and various control bits such as step, ebreakm, ebreaks, and ebreaku.

Exiting Debug Mode is initiated by the debugger through the dmcontrol register. The debugger must clear the haltreq bit and set the resumereq bit. Once the Debug Module confirms that the hart is ready (via dmstatus), the hart resumes normal instruction execution from the address stored in dpc.

If an invalid instruction is executed in the program buffer, or if an exception occurs during abstract command execution, the hart remains in Debug Mode, and the corresponding error code is updated in the cmderr field of the abstractcs register. This ensures that faulty debug operations do not unintentionally resume program execution or corrupt state.

This robust mechanism for entering, managing, and exiting Debug Mode makes the RISC-V debug architecture suitable for a wide range of use cases—from deeply embedded systems requiring minimal overhead to complex multi-core SoCs needing advanced debugging and trace capabilities.

1.10 Target Specification

Target specifications formulated for the design are:

- 32-bit, 5 stage Pipelined Processor
- Supports RV32G (RV32IMAFD) instructions
- Integrated Floating Point Unit
- Split I and D (Instruction and Data) caches
- Virtual Memory
- Data and Instruction Translation Lookaside Buffers (DTLB & ITLB)
- Branch Prediction Unit
- System Counters
- Exception and Trap controller
- CLINT Interrupt controller for timer and software interrupts
- ZiCSR extension support
- Debug Support
- PMP support for security applications

1.11 Conclusion

In the above study, we have compared various industrial and academic RISC-V designs. The RISC-V ISA was thoroughly studied to understand the interaction between software and hardware, allowing us to identify areas where hardware can be enhanced to handle certain anomalies—such as hazards and stalls—more efficiently, thereby reducing the compiler’s burden and accelerating the software development process. Building on the insights gained from literature and prior designs, we proposed a new RISC-V architecture incorporating several critical design blocks and features. The inclusion of features such as CSR, exception and interrupt handling, debug support, memory protection, TLB-based address translation, and cache management will make this processor suitable for academic research and low-power embedded control applications. Furthermore, the design will progress toward an ASIC implementation, enabling more efficient, compact, and power-optimized deployment in real-world embedded systems.